

IE University

Vehicle Insurance Fraud Detection

AOI UENO
ESTELLE ARNANDER
JOSÉ BOUZA FEMENIA
LORAINÉ NIETO
MANVI GUPTA
SEIF OMARA

Course: AI-ML LEARNING FOUNDATIONS
Professor: MATTEO TURILLI

Github: <https://github.com/1manvi/ML-Vehicle-Insurance-Fraud-2026>

Dataset: [Angoss Vehicle Insurance Fraud](#)

1. Problem Statement

Vehicle insurance fraud creates major financial losses for insurers and increases premiums for legitimate customers. Detecting fraudulent claims is difficult because the dataset is highly imbalanced, only around 6% of claims are fraudulent, while the remaining 94% are legitimate. This means that a model predicting every claim as non-fraudulent could still achieve very high accuracy while failing to detect fraud cases.

This project investigates how different imbalance handling strategies affect fraud detection performance. Three resampling strategies (no resampling, random oversampling, and SMOTE) were compared across three machine learning models, Logistic Regression, Random Forest, and XGBoost, creating a 3×3 experimental framework.

The task was framed as a binary classification problem focused on balancing fraud detection recall and false positives. Because of the severe class imbalance, PR-AUC was selected as the primary evaluation metric instead of accuracy.

2. Dataset Summary

The project used the Vehicle Insurance Fraud Detection dataset from Angoss Knowledge Seeker, publicly available on Kaggle. The dataset contains 15,420 insurance claims and 32 features related to claimant information, policy details, vehicle characteristics, and accident conditions. The target variable, FraudFound, is binary, where 1 represents fraud and 0 a legitimate claim.

Only 923 observations correspond to fraudulent claims, representing approximately 6% of the dataset. This imbalance creates a challenging classification problem because models tend to favour the majority class.

The dataset also contains many categorical variables, including nominal, binary, and ordinal features, requiring different encoding strategies during preprocessing. Exploratory analysis showed weak linear correlations between individual variables and the target, suggesting that fraud detection depends more on feature interactions than on single highly predictive variables

3. Pipeline Overview

3.1 Preprocessing

The dataset was split into training, validation, and test sets using a stratified 70/15/15 split to preserve the original fraud distribution. The Age feature contained invalid zero values, which were treated as missing and imputed using training-set median values to avoid data leakage. Several leakage-risk columns were also removed before modelling.

Different encoding strategies were applied depending on feature type: ordinal encoding for ordered categories, label encoding for binary variables, and one-hot encoding for nominal features. Numerical variables were scaled when required, particularly for Logistic Regression.

To address class imbalance, three training conditions were evaluated for each model: no resampling, random oversampling, and SMOTE. Resampling was applied only to the training folds during model training.

3.2 Feature Engineering

Several domain-inspired features were created to improve the models' ability to identify suspicious claims. A `high_claim_history` feature was introduced to identify customers with a large number of past claims, as repeated claims may indicate abnormal behaviour. A `weekend_accident` feature captured whether the accident occurred during weekends, based on the assumption that accident patterns may differ between weekdays and weekends.

Vehicle-related features were also engineered. An `old_vehicle` indicator was created because older vehicles may show different fraud behaviour than newer ones. A `high_deductible` feature was included to capture unusual deductible patterns, while a `multiple_cars` feature identified policyholders insuring several vehicles simultaneously.

These engineered variables were designed to incorporate simple domain knowledge into the modelling process and help capture interactions that may not be directly represented by the original variables alone.

3.3 Modelling Approach

The project followed a 3×3 experimental design where Logistic Regression, Random Forest, and XGBoost were evaluated under three imbalance handling strategies: no resampling, random oversampling, and SMOTE.

Logistic Regression served as the baseline model, Random Forest as a non-linear ensemble method, and XGBoost as the most advanced boosting model. Models were evaluated primarily using PR-AUC, alongside recall, precision, F1-score, and ROC-AUC.

All models used balanced class weighting where appropriate. Logistic Regression and Random Forest were additionally analysed using threshold tuning. XGBoost received the most extensive hyperparameter optimisation through RandomizedSearchCV followed by manual fine-tuning. The final threshold selection was performed on the validation set, while the test set was reserved for final evaluation only.

4. Key Results

4.1 Model Comparison Table

The results showed that both model choice and imbalance handling strategy strongly affected fraud detection performance. XGBoost combined with random oversampling achieved the best overall results, particularly in terms of PR-AUC and recall. Random Forest also produced competitive ranking performance, while Logistic Regression provided a stable baseline.

Random oversampling generally improved recall by exposing models to more fraud examples during training. In contrast, SMOTE often reduced performance, especially in the tree-based models. Several models achieved relatively high ROC-AUC values while still producing low recall at the default threshold, highlighting the importance of threshold tuning in imbalanced classification problems.

Overall, handling class imbalance effectively had a larger impact on performance than increasing model complexity alone.

4.2 Logistic Regression findings

Logistic Regression served as the baseline model and produced relatively stable results across all resampling conditions. Without resampling, the model struggled to detect fraudulent claims consistently because it remained heavily influenced by the majority class.

Random oversampling improved recall by exposing the model to a more balanced training distribution, while SMOTE produced mixed results without consistent PR-AUC improvements. Although Logistic Regression was less capable of modelling complex feature interactions than Random Forest or XGBoost, the performance gap was smaller than expected in several experiments. Threshold tuning also significantly improved recall, although at the cost of lower precision.

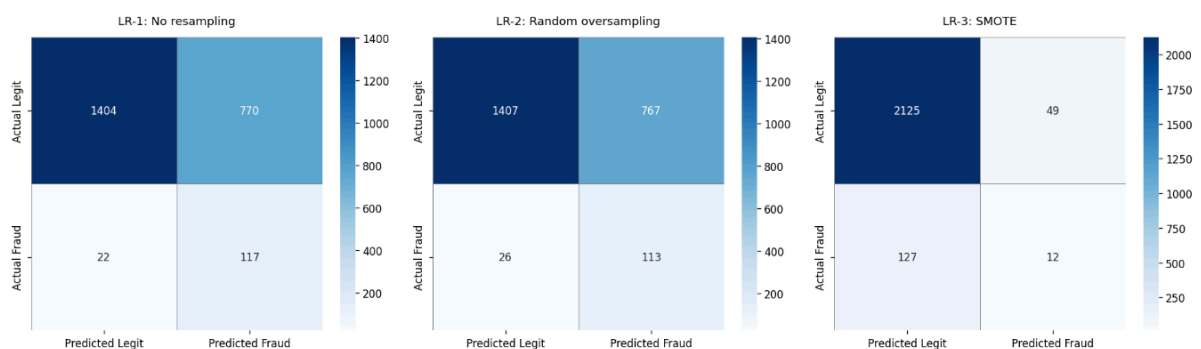


Figure 1: Confusion Matrix Comparison — Logistic Regression

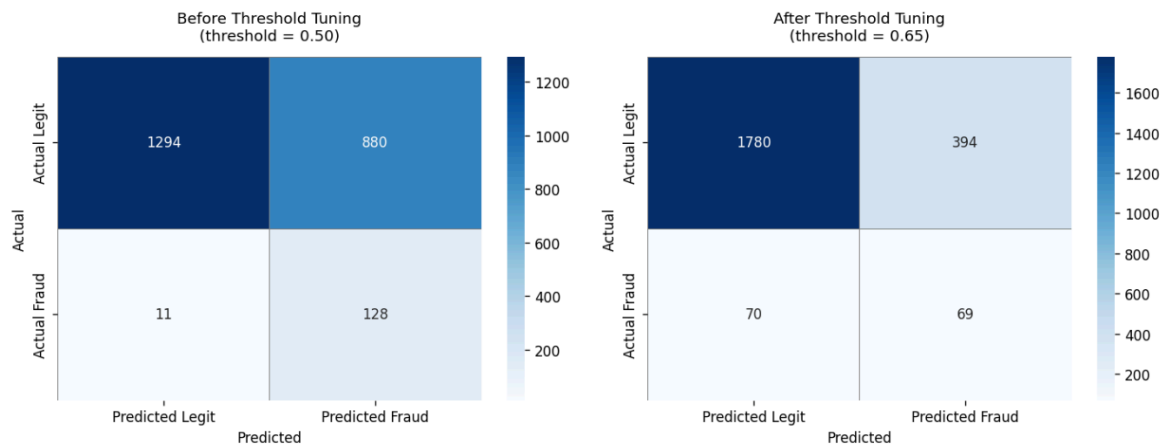


Figure 2: LR-1 Confusion Matrix Comparison — Before vs After Threshold Tuning

4.3 Random Forest findings

Random Forest achieved strong overall ranking performance and competitive ROC-AUC values across the experiments. The model captured non-linear feature interactions more effectively than Logistic Regression, particularly when combining categorical and engineered variables.

The baseline configuration achieved the strongest PR-AUC among the Random Forest variants, although recall remained low at the default threshold. Random oversampling improved recall and F1-score, while SMOTE produced the weakest results, likely because synthetic samples reduced the realism of the training distribution.

Threshold tuning substantially improved recall while maintaining acceptable precision. Overall, Random Forest produced stable and interpretable results but was ultimately outperformed by the tuned XGBoost models.

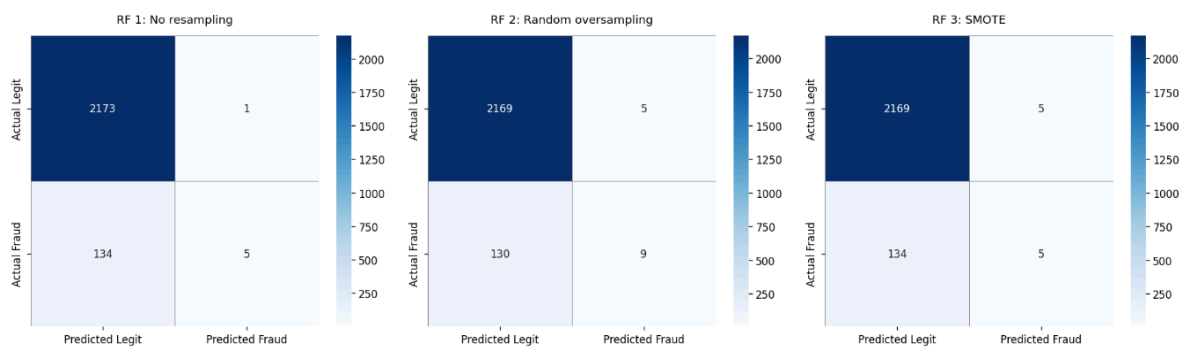


Figure 3: Confusion Matrix Comparison — Random Forest

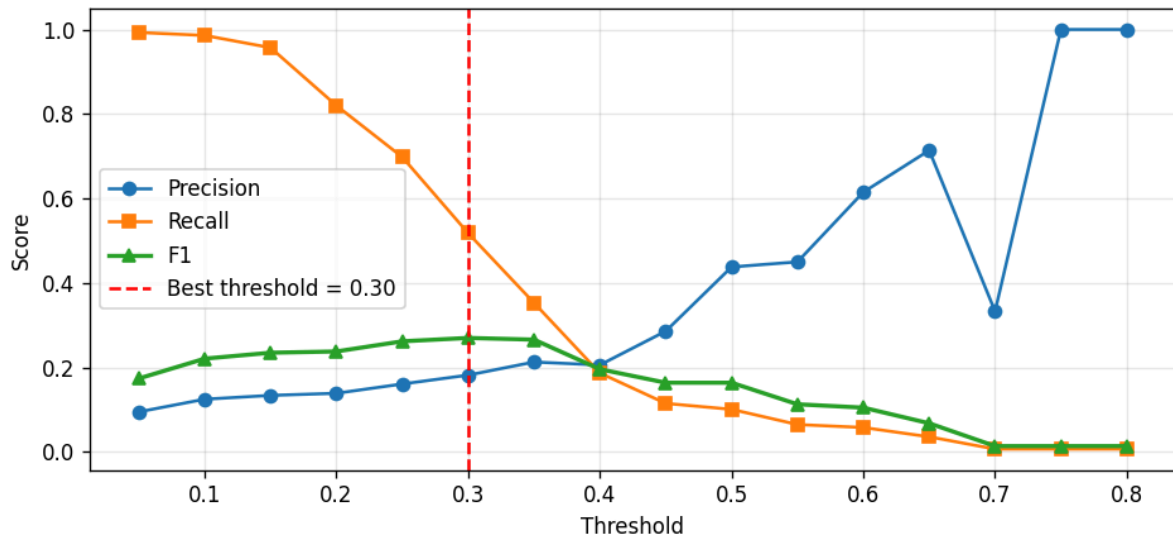


Figure 4: RF-1 Tuned — Threshold vs Metrics

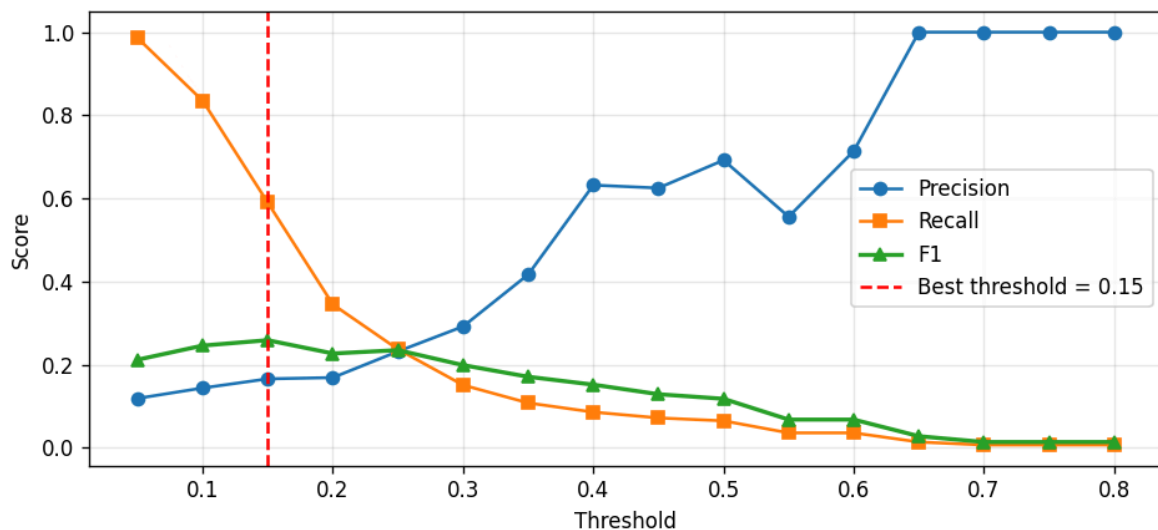


Figure 5: RF-2 Tuned — Threshold vs Metrics

4.4 XGBoost findings

XGBoost produced the strongest overall fraud detection performance, particularly when combined with random oversampling. The model achieved the best PR-AUC scores and was effective at ranking fraudulent observations above legitimate claims.

In contrast, the SMOTE configuration performed poorly, with recall collapsing despite acceptable ROC-AUC values. This suggests that synthetic samples generated through interpolation were not realistic for this categorical-heavy dataset and negatively affected model learning.

Hyperparameter tuning produced only modest improvements compared to the baseline configuration, indicating that performance was constrained more by dataset signal than parameter optimisation. Threshold tuning had a larger practical impact, significantly

improving recall while maintaining acceptable precision. Overall, the experiments showed that imbalance handling and threshold selection mattered more than model complexity alone.

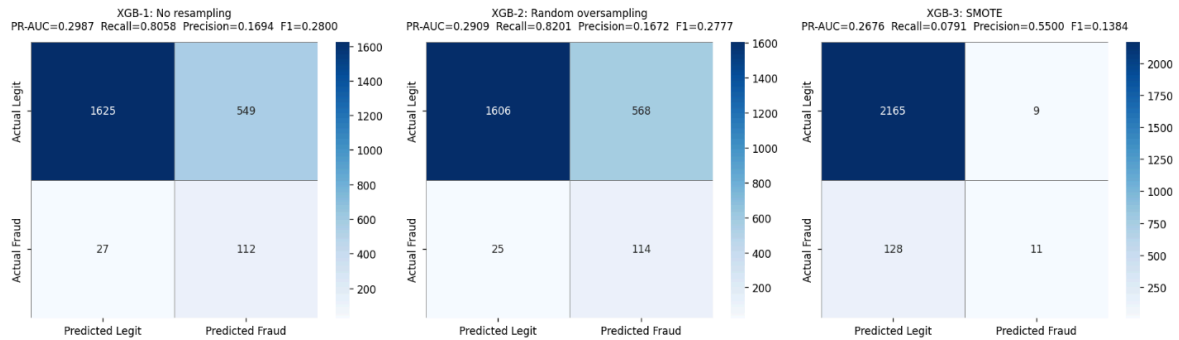


Figure 6: Confusion Matrix Comparison — XGBoost

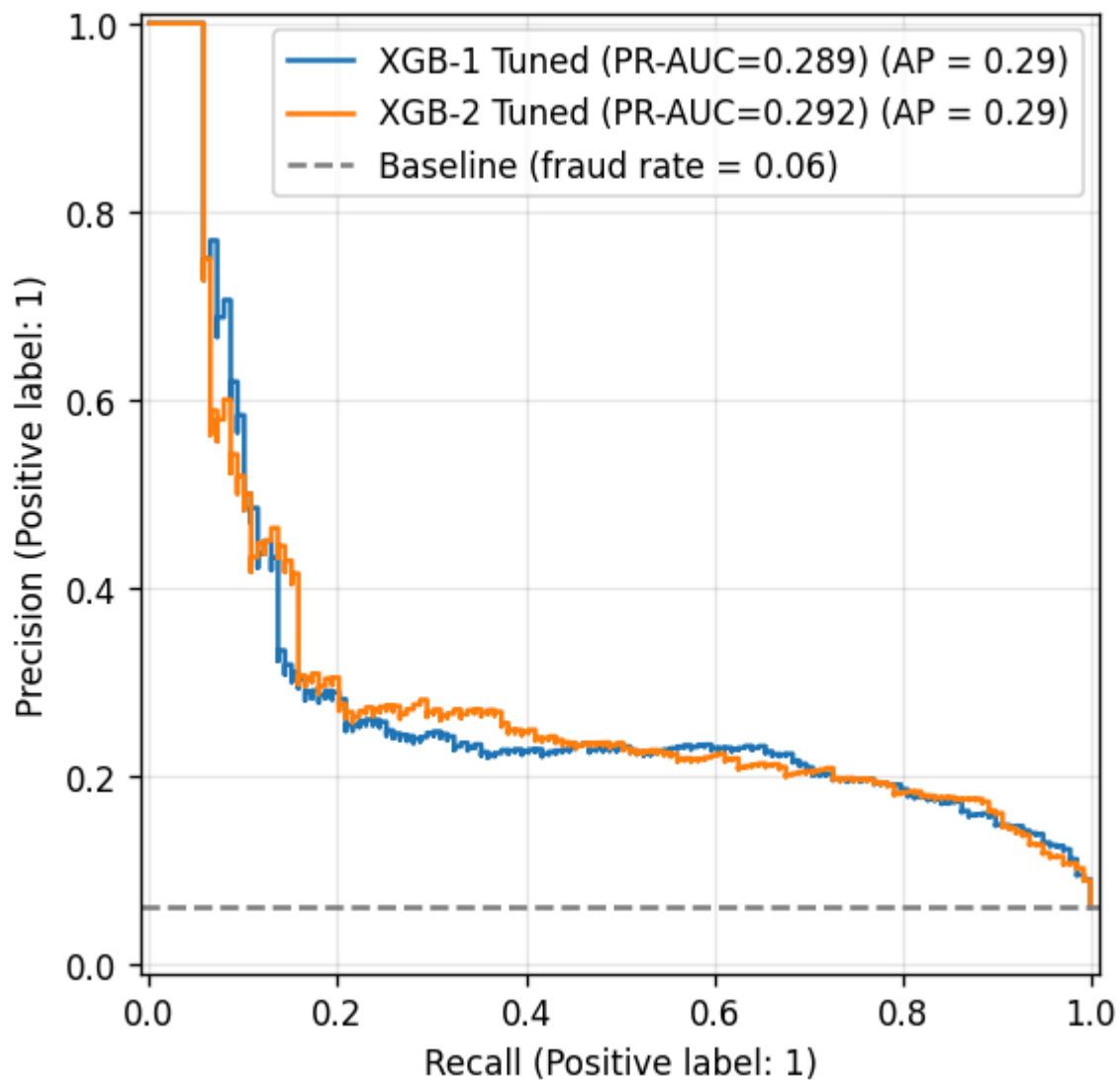


Figure 7: Precision-Recall Curves — Tuned Models

5. Reflection on Failures and Limitations

One of the main findings of the project was that more complex techniques did not always improve performance. In particular, SMOTE consistently underperformed in the tree-based models, especially XGBoost. A likely explanation is that SMOTE generates synthetic observations through interpolation between existing samples, which may not work well for datasets containing many categorical variables. After encoding, these interpolated values can create unrealistic feature combinations that do not represent genuine fraud patterns, ultimately reducing the model's ability to generalise effectively.

Another important observation was that hyperparameter tuning produced only modest improvements compared to the impact of threshold tuning and imbalance handling strategies. While tuning slightly improved PR-AUC and probability calibration, the gains remained relatively small overall. This suggests that model performance was constrained more by the predictive signal available in the dataset than by parameter optimization itself. In other words, increasing model complexity alone could not fully overcome the limitations of the data.

The experiments also showed that strong ROC-AUC performance does not necessarily translate into effective fraud detection at the default classification threshold. Several models were able to rank fraudulent claims reasonably well while still producing very low recall values when using the standard 0.5 cutoff. This highlighted the importance of threshold tuning in highly imbalanced classification tasks, where the practical objective is often to maximise fraud detection rather than overall classification accuracy.

Another limitation of the project is that the dataset was relatively structured and limited in scope compared to real-world fraud detection systems. Real insurance fraud investigations often include additional information such as claim histories, text descriptions, images, or behavioural patterns collected over time. Incorporating these types of data could potentially improve fraud detection performance beyond what was achieved in this project.

The experiments were also conducted using a single dataset with a fraud rate of approximately 6%, meaning the findings may not generalise to other fraud detection environments with different customer populations or fraud behaviours. Additionally, some demographic patterns present in the dataset could introduce bias into the models, raising fairness concerns in real-world insurance applications where automated fraud detection systems may disproportionately affect certain groups of claimants.

Finally, although the final evaluation was performed on a held-out test set, some early experimentation stages involved checking test-set metrics before the modelling process was completely finalised. While this did not fundamentally affect the conclusions of the project, it may have introduced a small degree of optimism into the reported results.

6. Group Collaboration

People	Model ownership	What to build & commit
Manvi & Estelle	Logistic Regressionx3 resampling conditions	logistic_regression.py LR-1: no resampling (use class_weight='balanced') LR-2: random oversampling LR-3: SMOTEImport evaluate_model() from utils.py for all three.
Aoi & Loraine	Random Forestx3 resampling conditions+ EDA	random_forest.py + eda.py RF-1: no resampling RF-2: random oversampling RF-3: SMOTEEDA: class imbalance plot, feature distributions,correlation heatmap, missing values, encoding decisions.
Jose & Seif	XGBoostx3 resampling conditions	xgboost_model.py XGB-1: no resampling XGB-2: random oversampling XGB-3: SMOTE + Report

References (not counted toward word limit)

- Dataset: Angoss Knowledge Seeker, Vehicle Insurance Fraud Detection
- Burkov, A. (2020). *Machine Learning Engineering*. True Positive Inc.
- scikit-learn, xgboost, imbalanced-learn library references